Laboratory Work 3: "Navigation in Jetpack Compose"

Variant 5: "My Recipes"

Student: Alexander Sigolaev

Group: AI-234

Development environment: Android Studio, Kotlin, Jetpack Compose

Date: May 8, 2026

1. Aim of the Work

The aim of this laboratory work is to develop an Android application with three screens and a navigation system in Jetpack Compose. The application must demonstrate route-based screen transitions, argument passing, ViewModel-managed data, and back navigation with popBackStack().

2. Assignment and Variant

According to variant 5, the application topic is My Recipes. The application contains a recipe list, a recipe creation/editing form, and a recipe details screen. The details screen receives a recipe ID through the route details/{recipeId}.

- RecipesListScreen displays all recipes with the recipe name and cooking time.
- AddRecipeScreen contains fields for the recipe name, ingredients, instructions, cooking time, and difficulty.
- DetailsRecipeScreen displays the selected recipe and allows changing difficulty, editing, deleting, and going back.
- Recipe data is managed by RecipeViewModel.

3. Project Structure

```
com.sigolaev.lab3
|-- MainActivity.kt
|-- model
|   |-- Recipe.kt
|-- viewmodel
|   |-- RecipeViewModel.kt
|-- ui
|   |-- RecipeNavHost.kt
|   |-- RecipeScreens.kt
|   |-- theme
```

4. Implementation

4.1 Data Model

The Recipe data class stores all required information: title, ingredients, instructions, cooking time, and difficulty. Difficulty is represented as an enum with Easy, Medium, and Hard values.

```
data class Recipe(
    val id: Int,
    val title: String,
    val ingredients: String,
    val instructions: String,
    val cookingTimeMinutes: Int,
    val difficulty: Difficulty = Difficulty.EASY
)

enum class Difficulty {
```

4.2 ViewModel

RecipeViewModel keeps the recipe list in StateFlow objects and exposes functions for adding, editing, deleting, searching, and changing difficulty. This keeps business logic outside the composable screens.

```
fun addRecipe(
    title: String,
    ingredients: String,
    instructions: String,
    cookingTimeMinutes: Int,
    difficulty: Difficulty
) {
    val recipe = Recipe(
        id = nextId++,
        title = title,
        ingredients = ingredients,
        instructions = instructions,
        cookingTimeMinutes = cookingTimeMinutes,
        difficulty = difficulty
    )
    _recipes.value = listOf(recipe) + _recipes.value
    publishState()
}
```

4.3 Navigation

The application uses Navigation Compose. The list screen is the start destination. The details route contains the required recipeId argument, and the add route supports an optional recipeId for editing.

```
private object RecipeRoutes {
    const val LIST = "recipes"
    const val ADD = "add?recipeId={recipeId}"
    const val DETAILS = "details/{recipeId}"

    fun add(recipeId: Int? = null): String =
        recipeId?.let { "add?recipeId=$it" } ?: "add"

    fun details(recipeId: Int): String = "details/$recipeId"
}
```

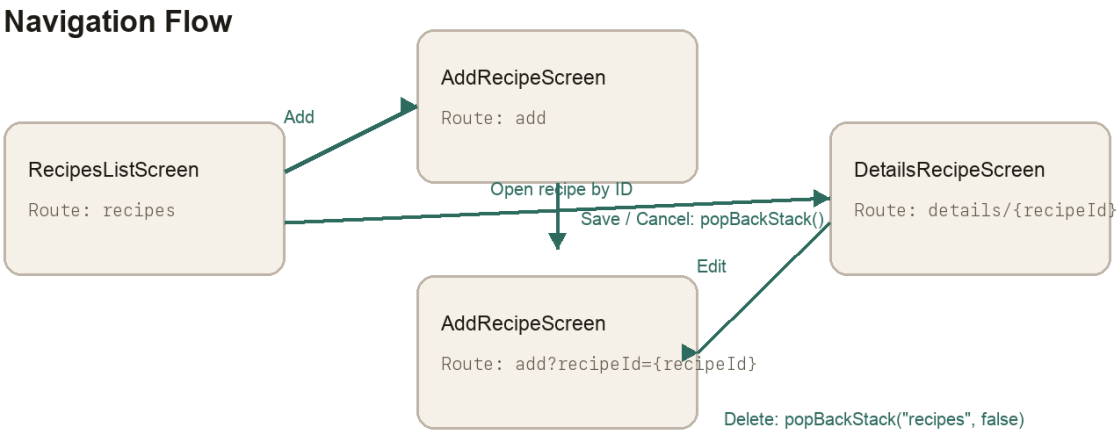

Figure 1 - Navigation flow of the application

4.4 User Interface

The visual style follows the Lab 2 application: Material 3 components, a warm custom theme, clear cards, top app bars, floating action button, and compact controls. All visible text is in English.

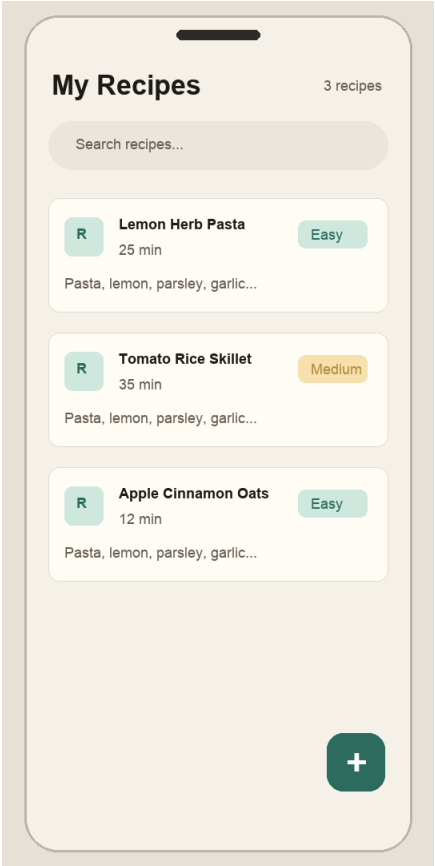

Figure 2 - Recipes list screen

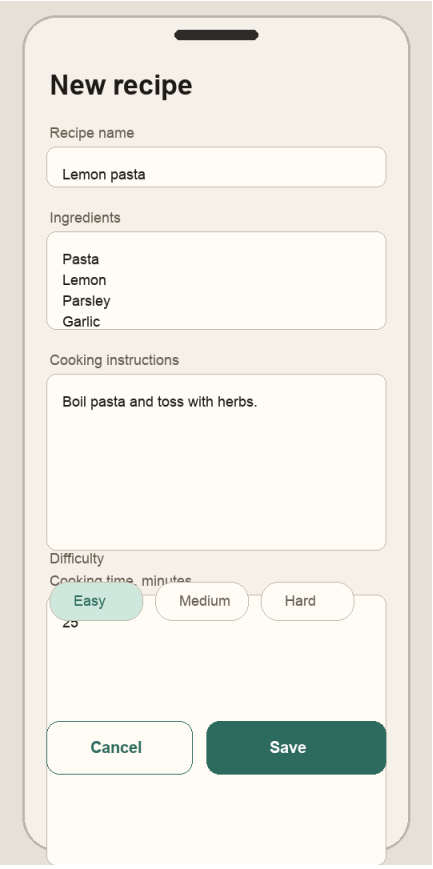

Figure 3 - Add recipe screen

Figure 4 - Recipe details screen

5. Testing and Build Verification

The following checks were performed successfully:

- ./gradlew testDebugUnitTest
- ./gradlew assembleDebug
- The debug APK was generated successfully in app/build/outputs/apk/debug/app-debug.apk.

6. Conclusion

During this laboratory work, an Android recipe application was created using Kotlin, Jetpack Compose, Material 3, ViewModel, StateFlow, and Navigation Compose. The application implements all required screens for variant 5, passes the selected recipe ID through details/{recipeId}, supports add/edit/delete operations, and uses popBackStack() for returning between screens.